

Procesos

¿Qué es un proceso?

Uno de los conceptos clave en un sistema operativo es el **proceso**.

Un **programa** es simplemente un conjunto de instrucciones.

Definición de proceso:

Un **proceso** es una **abstracción** que representa cada ejecución de un programa.

- **Importante:** un proceso no tiene que estar ejecutándose constantemente. Su **ciclo de vida** pasa por varias etapas, y la **ejecución** es sólo una de ellas.

Concepto de proceso

Podemos entender un proceso como:

- Cada instancia de un programa en ejecución.
- La entidad que el sistema operativo crea para ejecutar un programa.

Un proceso está formado por:

- **Espacio de direcciones:** conjunto de direcciones que el proceso puede usar.
- **Punto de control:** indica la siguiente instrucción que será ejecutada.

Algunos sistemas permiten que un proceso tenga **más de un punto de control**, lo que implica que el proceso puede ejecutar varias partes de su código al mismo tiempo. A esto se le llama **hilos (threads)**

Ejemplo práctico

- Si abrimos dos terminales y ejecutamos `ls -lR /` en una y `ls -lR /usr` en otra, tenemos **dos procesos distintos** ejecutando el **mismo programa** (`/bin/ls`).
- Cada uno ejecuta de forma **secuencial**; pero el sistema operativo va **alternando** entre ellos.
- Aunque parece que se ejecutan a la vez (**conurrencia**), en realidad sólo **uno se ejecuta en cada instante**. Esto se denomina **multitarea (multitasking)**.

En sistemas con **varias CPUs**, sí es posible tener **ejecución en paralelo real**, pero **cada CPU sólo ejecuta un proceso a la vez**.

Normalmente, el **número de procesos activos es mayor que el número de CPUs**

disponibles. También existen procesos que son **concurrentes internamente**, mediante el uso de **hilos de ejecución**.

Estructura básica del espacio de direcciones de un proceso

El espacio de direcciones de un proceso se divide, en su forma más simple, en tres regiones:

- **Código:** contiene el código de todas las instrucciones del programa.
- **Datos:** incluye las variables globales y la memoria asignada dinámicamente (heap), que generalmente forma parte de esta región.
- **Pila:** se utiliza para:
 - El paso de parámetros
 - Guardar la dirección de retorno cuando se llama a una función.
 - Almacena variables locales de la función en ejecución.

Regiones adicionales

Además de estas tres, pueden existir otras regiones como:

- **Memoria compartida**
- **Librerías dinámicas enlazadas.**
- **Ficheros mapeados en memoria**, entre otros.

Para visualizar el espacio de direcciones de un proceso en Unix, se puede usar el comando `pmap`.

- Este comando muestra las **direcciones virtuales** (también llamadas **lógicas**) que usa el proceso.
- El espacio tiene "huecos", y **no refleja direcciones físicas reales** de memoria.

Procesos en UNIX

¿Qué es UNIX?

UNIX es un término genérico que se refiere a una familia de sistemas operativos. Existen muchas variantes o "sabores" de UNIX, tanto comerciales como libres, por ejemplo Linux, Solaris, AIX o FreeBSD. Cada variante puede o no cumplir con distintos estándares que regulan su funcionalidad, implementación o interfaz con el sistema operativo. Los principales estándares son: System V, BSD, POSIX.

¿Qué es el kernel?

El kernel es el núcleo del sistema operativo, el programa principal que controla el hardware y permite ejecutar procesos. Este núcleo reside en un archivo específico (como /unix, /vmlinuz, /bds, etc.) que se carga durante el arranque del sistema mediante un programa llamado bootloader.

El kernel se encarga de:

- Inicializar el sistema y crear el entorno necesario para ejecutar los procesos.
- Crear algunos procesos iniciales, entre ellos INIT (con PID 1), que es el primer proceso de usuario y el antecesor de todos los procesos de usuario en el sistema.
- Gestionar la interacción directa con el hardware.
- Proporcionar la interfaz de llamadas al sistema (system calls) que los procesos de usuario usan para interactuar con el kernel y, por tanto, con el hardware.

¿Cómo arranca una máquina UNIX?

Al encender o reiniciar la máquina comienza el procesador de arranque, que consta de cuatro fases principales:

1. **Firmware de las placas base (MB firmware):** El microprocesador ejecuta el código en una dirección fija, donde se encuentra el firmware (BIOS, UEFI, OpenBoot, etc). Este firmware realiza una serie de comprobaciones básicas (POST) y busca un dispositivo desde donde arrancar.
2. **Bootloader:** Es un programa pequeño y sencillo que el firmware carga. Su función es cargar el kernel del sistema operativo en memoria. La configuración del bootloaders define dónde está el kernel y qué parámetros pasarle.
3. **Kernel:** El kernel se carga en memoria, inicializa los dispositivos, configura las tablas internas (procesos, archivos abiertos...) y crea el primer proceso de usuario (INIT o systemd en algunas distribuciones Linux).
4. **Servicios:** INIT (o systemd) lee su configuración y ejecuta scripts para iniciar servicios del sistema (red, montaje de sistemas de archivos, daemons, servicios de login, etc.) dejando el sistema listo para su uso.

El proceso de arranque es similar en otros sistemas operativos, aunque las herramientas y procesos específicos varían.

¿Qué hace el kernel?

El kernel es el único proceso que corre directamente sobre el hardware del sistema. Los procesos de usuario no interactúan con el hardware directamente, sino a través de llamadas al sistema que el kernel gestiona.

Además, el kernel atiende las solicitudes de los dispositivos externos mediante interrupciones,

que son señales enviadas al procesador para notificar eventos (por ejemplo, que una operación de E/S ha finalizado).

Modo kernel y modo usuario

Para funcionar correctamente, los sistemas operativos diferencian al menos dos modelos de ejecución:

- **Modo usuario:** Donde se ejecuta el código de los procesos de usuario.
- **Modo kernel:** Donde se ejecuta el código del sistema operativo.

Existen tres mecanismos principales que llevan a la ejecución en modo kernel:

1. **Llamadas al sistema:** cuando un proceso de usuario solicita un servicio del kernel.
2. **Excepciones:** situaciones anónimas como división por cero o errores de acceso a memoria.
3. **Interrupciones:** señales generadas por dispositivos hardware para avisar al kernel.

Algunas instrucciones del procesador sólo se pueden ejecutar en modo kernel.

Ejemplos de ejecución en modo kernel

Se puede observar el tiempo consumido en modo usuario y modo kernel con el comando `time`. Por ejemplo:

- Un programa que simplemente ejecuta un bucle infinito consume casi todo el tiempo en modo usuario.
- Si el programa hace llamadas al sistema (por ejemplo, `getpid()` en un bucle), el tiempo en modo kernel aumenta.
- Cuando un proceso se envía señales a sí mismo o a otros procesos, o provoca excepciones controladas, el tiempo en modo kernel crece notablemente.
- Interactuar con dispositivos (como mover el ratón o presionar teclas) también incrementa el tiempo en modo kernel.

Esto refleja que las operaciones que requieren interacción con el kernel o hardware se ejecutan en modo kernel, mientras que el resto del código del usuario se ejecuta en modo usuario.

Procesos e hilos (threads) en UNIX

En sistemas UNIX tradicionales, un proceso se define por:

- **Espacio de direcciones:** conjunto de direcciones de memoria que el proceso puede referenciar.

- Punto de control: Indica la siguiente instrucción que se va a ejecutar (contador de programa).

En sistemas UNIX modernos, un proceso puede tener varios puntos de control concurrentes, llamados **hilos**, que comparten el mismo espacio de direcciones pero pueden ejecutarse simultáneamente.

Espacio de direcciones

Los procesos utilizan direcciones virtuales. Parte de este espacio corresponde al código y datos del kernel (llamado espacio del sistema o espacio kernel), al que sólo se puede acceder en modo kernel.

El kernel mantiene:

- Estructuras de datos globales.
- Estructuras específicas por proceso
- Registros de memoria que apuntan al espacio de direcciones del proceso actualmente en ejecución.

Kernel UNIX

El kernel está implementado en lenguaje C y está formado por:

- **Código kernel**: ejecutado en modo kernel, incluye el código para llamadas al sistema, manejadores de interrupciones y excepciones.
- **Datos kernel**: variables globales accesibles a todos los procesos (tabla de procesos, tabla de inodos, tabla de archivos abiertos...).
- **Pila kernel**: memoria usada como pila cuando se ejecuta código en modo kernel, para pasar parámetros y almacenar variables locales.

Kernel reentrante

El kernel UNIX es reentrante, lo que significa que:

- Varios procesos pueden estar ejecutando funciones del kernel al mismo tiempo, incluso la misma función simultáneamente.
- Para esto, el código del kernel debe ser de sólo lectura.
- Los datos globales del kernel deben estar protegidos contra accesos concurrentes.
- Cada proceso dispone de su propia pila de kernel.

Protección de datos kernel

En kernels tradicionales no preemptibles: Un proceso en modo kernel no puede ser interrumpido hasta que termine, bloquee o vuelva a modo usuario. Sólo ciertas estructuras de datos deben protegerse con flags simples.

En kernels preemptibles modernos: Un proceso en modo kernel puede ser interrumpido si otro proceso de mayor prioridad está listo para ejecutarse. Por ello, todas las estructuras de datos del kernel deben protegerse mediante mecanismos más complejos, como semáforos, especialmente en sistemas multiprocesador.

Estructuras de Datos del Sistema Operativo

¿Qué necesita el sistema operativo para gestionar los procesos?

Para poder gestionar adecuadamente los procesos, el sistema operativo necesita llevar a cabo varias tareas fundamentales. En primer lugar, debe asignar memoria para que el programa se pueda cargar y ejecutar. Esta es la base para que cualquier proceso comience a funcionar.

Además, como varios procesos pueden estar ejecutando el mismo programa, el sistema operativo debe ser capaz de distinguirlos entre sí. Para ello, se utilizan estructuras como el **descriptor de proceso** o el **identificador de proceso**, que permiten reconocer de forma única a cada uno.

Cuando un proceso no se está ejecutando activamente, es necesario conservar toda la información relacionada con su estado de ejecución, esto incluye los registros del procesador, la memoria que está utilizando y los recursos que tiene asignados.

Estructuras de Datos del Sistema Operativo

El sistema operativo debe tener conocimiento de todos los procesos que existen en el sistema y del estado en que se encuentra cada uno. Para esto, se utilizan **listas de descriptores de procesos**, organizadas generalmente por estado, o incluso por cada dispositivo de entrada/salida involucrado.

Bloque de Control del Sistema (SCB)

El Bloque de Control del Sistema, es un conjunto de estructuras de datos utilizadas por el sistema operativo para gestionar la ejecución de todos los procesos presentes en el sistema.

Contenido habitual del SCB:

- Una lista de todos los descriptores de procesos.
- Un puntero al proceso que está siendo ejecutado actualmente en la CPU.

- Punteros a listas de procesos según su estado: listos para ejecutarse, bloqueados en espera de dispositivos, etc.
- Un puntero a la lista de descriptores de recursos asignados.
- Referencias a las rutinas de gestión de interrupciones de hardware y software, así como a las rutinas de manejo de errores.

¿Cuándo entra en funcionamiento el sistema operativo?

El sistema operativo se ejecuta en ciertas circunstancias específicas, entre las que se incluyen:

- **Cuando ocurre una excepción:** por ejemplo, cuando un proceso intenta realizar una operación no permitida, como dividir por cero o acceder a una dirección de memoria no válida.
- **Cuando un proceso realiza una llamada al sistema:** por ejemplo cuando un disco ha terminado una operación de escritura pendiente.
- **Cuando un dispositivo externo requiere atención (interrupción):** por ejemplo, cuando un disco ha terminado una operación de escritura pendiente.
- **De forma periódica,** mediante una interrupción del reloj, que permite al SO retornar el control y tomar decisiones de planificación.

Bloque de Control de Proceso (PCB)

¿Qué es el PCB?

El sistema operativo mantiene una **tabla de procesos**, en la que guarda información sobre cada uno de ellos. Cada entrada de esa tabla representa un solo proceso, y está compuesta por su correspondiente.

Bloque de Control de Proceso

El PCB es una estructura esencial que contiene todos los datos necesarios para que el sistema operativo pueda gestionar adecuadamente un proceso en particular.

Información contenida en el PCB

Identificador del proceso:

- Identificador único del proceso.
- Identificador del proceso padre (en caso de que el sistema operativo implemente jerarquía de procesos).
- Identificadores de usuarios y de grupo asociados.

Información para la planificación (scheduling)

- Estado actual del proceso (ejecutando, listo, bloqueado).
- Evento que espera (en caso de estar bloqueado).
- Parámetros de planificación: prioridad del proceso y otros datos relevantes para el algoritmo de planificación.

Regiones de memoria asignadas:

- Región de datos.
- Región de código.
- Región de pila.

Recursos asignados:

- Archivos abiertos, representados mediante una tabla de descriptores de archivos o "file handlers".
- Puertos de comunicación asignados.

Organización en listas:

- Punteros que permiten organizar los procesos dentro de listas (lista de procesos listos o bloqueados).

Comunicación entre procesos:

- Información sobre colas de mensajes, memoria compartida, semáforos, señales, etc.

Estructuras de datos en UNIX

Implementación de un proceso

Para implementar el concepto de proceso, UNIX utiliza diversas estructuras que permiten la ejecución de un programa. En esencia, un proceso está compuesto por varios componentes clave:

- **Espacio de direcciones del usuario:** Contiene el código del programa, sus datos, la pila, regiones de memoria compartida y ficheros mapeados.
- **Información de control:** que permite al núcleo gestionar llamadas al sistema y otras funciones internas.
- **Mapas de traducción de direcciones:** necesarios para traducir direcciones virtuales físicas.
- **Credenciales:** identifican al usuario que ejecuta el proceso, determinando sus privilegios.
- **Variables de entorno:** otra vía para pasar información al proceso.

- **Contexto hardware:** registra el estado del hardware, como los registros del procesador. Este contexto se guarda en una zona especial de la `u_area` llamada **PCB (Process Control Block)** cuando se produce un cambio de contexto.

Aunque algunos de estos elementos son conceptualmente distintos, en la práctica comparten implementación. Por ejemplo, la pila del núcleo suele formar parte de la `u_area` y ñas credenciales del proceso se almacenan dentro de la estructura `proc`.

En sistemas Linux modernos, las estructuras `u_area` y `proc` han sido reemplazadas por una única estructura llamada `task_struct`.

La estructura `proc`

El núcleo del sistema mantiene una tabla de procesos, conocida como *process table*, compuesta por múltiples estructuras `proc`. Esta tabla se encuentra en el espacio del núcleo (kernel space), y cada entrada representa un proceso en el sistema.

Lo más relevante de esta estructura es que **permanece accesible en todo momento**, incluso cuando el proceso no está siendo ejecutado activamente por la CPU. Su contenido incluye toda la información que el núcleo necesita para gestionar un proceso.

Contenido relevante de `proc`

- Identificador del proceso (PID).
- Dirección de la `u_area`.
- Estado del proceso.
- Punteros a colas de listos, espera, etc.
- Prioridad del proceso y datos relacionados.
- Canal de sueño (sleep channel).
- Información sobre señales (máscaras, estados).
- Información sobre gestión de memoria.
- Punteros para mantener la estructura `proc` en colas de procesos disponibles, zombies, etc.
- Punteros a estructuras de hash basadas en el PID.
- Información sobre la jerarquía de procesos (procesos padre, hijos...).
- Flags o indicadores de estado.

`u_area`

La estructura `u_area` contiene información que sólo es relevante cuando el proceso se está ejecutando en la CPU. Aunque conceptualmente reside en el espacio de usuarios, **sólo es accesible desde el núcleo**, y únicamente cuando el proceso está activo.

Siempre se sitúa en la misma dirección virtual, facilitando su acceso por parte del sistema operativo.

Contenido relevante a la `u_area`

- PBC
- Puntero a la estructura `proc`
- Parámetros y valores de retorno de llamadas al sistema.
- Información de señales y sus manejadores.
- Tabla de descriptores de ficheros del usuario.
- Punteros a los inodes del directorio raíz, directorio de trabajo y terminal asociada.
- La pila del núcleo del proceso.

Credenciales

Las credenciales permiten al sistema identificar qué acciones puede realizar un proceso, tanto sobre archivos como sobre otros procesos. Cada usuario del sistema se identifica mediante un número llamado **UID (user ID)** y cada grupo con un **GID (group ID)**.

El usuario especial **root** tiene UID 0, y dispone de privilegios completos: acceso a todos los ficheros, envío de señales a cualquier proceso, y capacidad de realizar llamadas al sistema privilegiadas.

Acceso a archivos

El acceso a un archivo está condicionado por:

- **El propietario del archivo**
- **El grupo asociado del archivo.**
- **Los permisos del archivo.**

Cuando un proceso intenta acceder a un archivo, el sistema compara sus credenciales con las del archivo:

1. Si el UID del proceso coincide con el del archivo, se aplican permisos de propietario.
2. Si no coincide el UID pero sí el GID, se aplican permisos de grupo.
3. Si no coincide ninguno, se aplican permisos para "otros".

Tipos de credenciales

Cada proceso posee **tres pares de credenciales**

- **Real:** Identifica al usuario original
- **Efectiva:** se usa para determinar los privilegios al acceder a archivos.
- **Guardada:** permite revertir cambios temporales en las credenciales.

Estas credenciales controlan también el envío y recepción de señales entre procesos, y se ven afectadas por las llamadas al sistema `setuid()` y `setgid()`.

Cambios en las credenciales

Existen tres mecanismos principales que pueden modificar las credenciales de un proceso:

1. `setuid(uid)`: cambia el UID efectivo de un proceso.
2. `setgid(gid)`: cambia el GID efectivo.
3. `exec()`: reemplaza el programa del proceso actual, pudiendo cambiar credenciales si el archivo ejecutado tiene ciertos permisos especiales.

También existen llamadas para consultar credenciales: `getuid()`, `geteuid`, `getgid()`, `getegid()`.

Sobre setuid y setgid

Estas llamadas pueden:

- Cambiar el UID o GID efectivo a su correspondiente valor real o guardado.
- Si el proceso tiene permisos de root (UID efectivo = 0), pueden cambiar las tres credenciales (real, efectiva y guardada).

Sobre exec()

Cuando un proceso ejecuta un programa con `exec()`, y el ejecutable tiene el bit *setuid* o *setgid* activado:

- **Setuid (s en los permisos del propietario)**: cambia el UID efectivo y guardado al UID del propietario del archivo.
- **Setgid (s en los permisos del grupo)**: cambia el GID efectivo y guardado al GID del archivo.

Variables de entorno

Las **variables de entorno** son cadenas de caracteres que permiten al sistema operativo y a los programas almacenar información relevante sobre el entorno de ejecución de un proceso. Son utilizadas ampliamente por shells, scripts, programas del sistema y aplicaciones para conocer datos como el directorio actual, la configuración de idioma, rutas a ejecutables, entre otros.

****Forma y ubicación**

- Las variables de entorno tiene la forma general de: **NOMBRE_VARIABLE=valor**

- Internamente, estas variables se almacenan al final de la **pila de usuario** en memoria.

Acceso a variables de entorno:

1. A través del tercer argumento de main:

La función main puede recibir un tercer argumento opcional: `int main (int argc, char *argv[], char *envp[])` Este tercer argumento, envp, es un array de punteros a cadenas que terminan en NULL. Cada cadena contiene una variable de entorno.

2. Usando la variable global environ:

También es posible acceder a las variables de entorno a través de una variable externa declarada como: `extern char **environ;` Al igual que envp, environ es un array de cadenas de caracteres finalizado con NULL.

3. Mediante funciones de la biblioteca estándar:

- `getenv("NOMBRE")`: Obtiene el valor de una variable.
- `putenv("NOMBRE=valor")`: Añade o modifica una variable.
- `setenv(nombre, valor, sobrescribir)` Añade o modifica de forma controlada.
- `unsetenv("NOMBRE")`: Elimina una variable de entorno.

Ciclo de vida de los procesos

Todo proceso en un sistema operativo comienza su existencia mediante una llamada al sistema para crear procesos. Esta función proporcionada por el sistema operativo, y el proceso que realiza esta llamada se convierte en el **proceso padre** del proceso recién creado.

Creación y ejecución

En algunos sistemas operativos, como Windows, la llamada para crear un proceso debe incluir también el archivo ejecutable que el nuevo proceso va a ejecutar. Sin embargo, en los sistemas tipo Unix, la creación de un proceso y la ejecución de un programa son operaciones separadas que se realizan mediante llamadas al sistema diferentes, como `fork()` y `exec()`.

Estados de un proceso

A lo largo de su vida, un proceso puede pasar por varios estados. Los principales son:

- **Ejecutándose (running)**: el proceso está siendo ejecutado actualmente por la CPU.
- **Listo para ejecutar (ready to run)**: el proceso está preparado para ejecutarse pero está esperando que la CPU lo seleccione.
- **Bloqueado**: el proceso está esperando un evento externo, como la finalización de una operación de entrada/salida.

Cuando finaliza, un proceso realiza una llamada al sistema para terminar su ejecución. Este es el punto final de su ciclo de vida, y se encarga de liberar los recursos asignados al proceso.

Sistemas con intercambio (swapping)

Sistemas antiguos: intercambio completo

El sistemas más antiguos, parte de la memoria secundaria (como el disco) se utilizaba para **intercambiar procesos completos** fuera de la memoria principal con el objetivo de aumentar el grado de multiprogramación (número de procesos activos al mismo tiempo). Esta técnica se conoce como **swapping**, y Cuando un proceso era intercambiado fuera de la memoria, pasaba al estado de **intercambiado** o **suspendido**. Las transiciones asociadas a este estado son:

- **Swap in:** el proceso se vuelve a cargar en la memoria principal.
- **Swap out:** el proceso es desplazado fuera de la memoria.

Los sistemas que implementaban esta técnica se conocen como **sistemas con intercambio**.

Sistemas modernos: intercambio por páginas

En los sistemas actuales, el intercambio se realiza **por piezas** de los procesos, en lugar de intercambiar el proceso completo. Estas piezas suelen ser páginas de memoria. De esta forma, es posible ejecutar procesos que no están completamente cargados en memoria o incluso procesos más grandes que la propia memoria física disponible.

El espacio en disco utilizado puede ser una **participación dedicada** o un archivo de intercambio dentro del sistema de archivos. Este modelo se basa en el concepto de **memoria virtual**, y los sistemas que lo utilizan se conocen como **sistemas con paginación**.

Estados de los procesos

La terminología común utilizada para describir los estados de un proceso incluye:

- **En ejecución:** el proceso está usando la CPU.
- **Listo:** el proceso puede ejecutarse tan pronto como se le asigne CPU.
- **Bloqueado:** el proceso espera que ocurra un evento externo.
- **Intercambiado:** el proceso ha sido desplazado a la memoria secundaria, ya sea desde el estado listo o bloqueado.

Transiciones de estado

Entrada al estado de ejecución

El proceso que está primero en la cola de listos es seleccionado por el planificador para ejecutarse.

Entrada al estado listo para ejecutar

1. Se ha creado un proceso nuevo.
2. Un proceso que estaba ejecutándose es interrumpido por el sistema operativo mediante una conmutación de contexto (preempted).
3. Un proceso bloqueado se desbloquea cuando ocurre el evento esperado (con la finalización de una entrada/salida). Esto se llama **desbloqueo** o **despertar**.
4. Un proceso intercambiado (en estado listo) es traído de nuevo a memoria principal.

Entrada al estado bloqueado:

- Desde ejecución: el proceso hace una llamada al sistema que no puede completarse de inmediato y se bloquea.
- Desde bloqueado/intercambiado: el sistema operativo puede decidir traer a memoria un proceso bloqueado que estaba intercambiado, aunque no todos los sistemas permiten esta transición.

Un proceso también puede pasar a los estados bloqueado/intercambiado o list/intercambiado si el sistema operativo decide liberarlo de la memoria principal para hacer espacio a otros procesos.

Creación de procesos

La **creación de procesos** es un servicio del sistema operativo. Cuando un proceso invoca la llamada al sistema correspondiente, el sistema operativo realiza las siguientes acciones:

1. Asigna un identificador único al nuevo proceso.
2. Crea e inicializa su PCB, la estructura de datos que contiene toda la información del proceso.
3. Actualiza el SCB, añadiendo el nuevo proceso a su tabla.
4. Asigna memoria y, si es necesario, carga en ella el programa que se va a ejecutar.
5. Coloca el nuevo proceso en la cola de listos.

Modelos de creación

Un proceso puede crearse en diversos contextos, aunque internamente siempre se utiliza la misma llamada al sistema:

- **Inicialización del sistema:** durante el arranque del sistema se crean múltiples procesos.
 - Algunos interactúan directamente con el usuario.
 - Otros no tienen interacción directa y se encargan de servicios del sistema. Estos se conocen como **demonios (daemons)**
- **Llamadas del propio código del proceso:** por ejemplo, un programa que crea nuevos procesos para rellenar un búfer con datos.

- **Acción explícita del usuario:** al ejecutar un comando en una terminal o hacer click sobre un icono gráfico, es el código del entorno el que crea el nuevo proceso.
- **Procesamiento por lotes (batch).**

Terminación de procesos

Cuando un proceso finaliza, lo hace mediante una llamada al sistema que invoca el servicio de **terminación de proceso**.

Acciones del sistema operativo al finalizar un proceso:

- Elimina el PCB del proceso.
- Libera todos los recursos que estaban asignados al proceso.
- Si el proceso tiene **procesos hijos**, puede optar por:
 - Esperar a que estos terminen.
 - Terminarlos explícitamente.
 - Dejarlos ejecutarse independientemente.

Tipos de terminación

Existen dos formas principales de finalización:

1. **Terminación normal:** el proceso realiza de una llamada a la función de terminación `exit()`.
2. **Terminación anormal:** no está prevista por el código del procesador. Puede deberse a errores, señales externas o fallos del sistema. En cualquier caso, el proceso es forzado a terminar.

Ciclo de vida de los procesos en UNIX

En UNIX, un **proceso** es una instancia de un programa en ejecución. El sistema operativo crea un proceso con el propósito de ejecutar un programa, proporcionando un entorno de ejecución que incluye un espacio de direcciones y uno o varios puntos de control para gestionarlo.

Cada proceso tiene una vida útil definida: **se crea mediante la llamada al sistema `fork()` o `vfork()` y termina cuando se invoca `exit()`**. Durante su ejecución, un proceso puede iniciar otro programa mediante alguna de las variantes de `exec()`.

Además, todo proceso en UNIX **tiene un proceso padre**, y puede tener uno o más **procesos hijos**. Esta estructura forma un árbol de procesos, cuyo ancestro común es usualmente el proceso `init`. Si un proceso padre termina antes que sus hijos, estos son adoptados por `init`.

Estados de los procesos (System V R2)

Un proceso puede encontrarse en distintos **estados**, que reflejan su situación en el sistema:

- **Idle**. El proceso está en creación, pero aún no está listo para ejecutarse.
- **Runnable**. Está listo y puede ser seleccionado por el planificador.
- **Asleep**. Está esperando que ocurra algún evento externo. Puede estar en memoria principal o en el área de intercambio (swap).
- **User running**. El proceso se está ejecutando en modo usuario.
- **Kernel running**. El proceso se está ejecutando en modo núcleo.
- **Zombie**. El proceso ha finalizado pero su proceso padre aún no ha recogido su estado con una llamada a `wait()`, por lo que su estructura interna (`proc`) no ha sido liberada

Desde la versión 4.2BSD se introdujo un nuevo estado: **stopped**, que puede ser:

- **stopped-runnable**
- **stopped-blocked**

Este estado se alcanza cuando el proceso recibe señales como:

- SIGSTOP: detiene el proceso sin posibilidad de ignorarla
- SIGTSTP: se activa con ctrl+z, deteniendo el proceso en primer plano.
- SIGTTIN y SIGTTOU: cuando un proceso en segundo plano intenta leer o escribir en el terminal.

Para reanudar un proceso se usa SIGCONT.

Además, existen comandos de terminal para gestionar estos procesos:

- `jobs`: lista los trabajos del terminal.
- `fg`: mueve un trabajo al primer plano.
- `bg` lo envía al segundo plano.

Cambios de estado del proceso

- La ejecución **siempre comienza y termina en modo kernel**.
- La transición a estado **bloqueado** ocurre desde el modo kernel.
- Las transiciones hacia o desde el estado **runnable** también se gestionan en modo kernel.
- Al terminar un proceso, entra en estado zombie hasta que el proceso padre realice una llamada a `wait()`.

Llamadas al sistema

`fork()`

La llamada a `fork()` crea un nuevo proceso: **un clon del proceso padre**, con una copia exacta de su espacio de direcciones. La única diferencia entre padre e hijo es el valor que devuelve `fork()`:

- En el proceso hijo devuelve 0
- En el proceso padre devuelve el PID del hijo.

`exec()`

Una vez creado un proceso, este puede reemplazar su contenido con otro programa usando alguna variante de `exec()`. Este reemplazo afecta completamente al código, datos, pila y demás elementos del espacio de direcciones del proceso.

Es importante remarcar que `exec()` **no crea un nuevo proceso**. El proceso que invoca `exec()` sigue siendo el mismo con el mismo PID, pero con un contenido completamente diferente.

`exit()`

Cuando un proceso termina su ejecución, llama a `exit()`, que libera recursos y marca al proceso como zombie hasta que su padre recoja su estado.

Ejemplo de uso combinado de `fork()` y `exec()`

```
if ((pid=fork())==0)
{ // Proceso hijo
    if (execv("./programilla", args) == -1)
    {
        perror("fallo en exec");
        exit(1);
    }
}
else if (pid < 0)
{
    perror("fallo en fork");
}
else
{
    // Proceso padre continúa
}
```

Tareas relacionadas por `fork()`

1. Asigna espacio de intercambio (swap).
2. Asigna un PID y estructura `proc`.

3. Inicializa la estructura `proc`
4. Asigna mapas de traducción de direcciones.
5. Asigna y copia `u_area` desde el padre.
6. Actualiza campos en `u_area`.
7. Comparte la región de código del padre.
8. Duplica segmentos de datos y pila.
9. Inicializa el contexto hardware.
10. Cambia el estado del hijo a "listo para ejecutarse".
11. Devuelve 0 al hijo.
12. Devuelve el PID del hijo al padre.

Optimización de `fork()`

En muchos casos, un proceso hijo creado por `fork()` ejecuta inmediatamente un nuevo programa con `exec()`. Esto provoca que el trabajo hecho por `fork()` (como copiar datos y pila) se desaproveche, ya que `exec()` reemplaza todo el contenido del proceso.

Para optimizar esta situación, se introducen dos técnicas:

1. Copy-on-write (COW)

- El hijo y el padre **comparten** los datos y la pila.
- Estos se marcan como **sólo lectura**.
- Cuando uno de los procesos intenta modificarlos, se genera una excepción y **sólo se copia la página modificada**.

2. `vfork()`

- Utilizado cuando se planea usar `exec()` inmediatamente después de crear el proceso.
- El hijo **no copia nada**: ni datos ni pila.
- El hijo ejecuta en el espacio de direcciones del padre, que queda **bloqueado** hasta que el hijo llame a `exec()` o `exit()`.

Ejecutar programas con `exec()`

Al invocar `exec()`:

1. El espacio de direcciones actual se descarta.
2. Se carga el nuevo programa.
3. Se copian los argumentos y el entorno a la nueva pila de usuario.
4. Se reinician los manejadores de señales.
5. El contador de programa se sitúa en la primera instrucción del nuevo programa.

`exec()` no cambia el PID ni la estructura del proceso, pero si el ejecutable tiene activados los bits `setuid` o `setgid`, se modifican las credenciales efectivas del proceso.

Tareas realizadas por `exec`

1. Obtiene el archivo ejecutable.
2. Verifica los permisos de ejecución.
3. Comprueba que es un archivo ejecutable válido.
4. Si empieza por `#!`, invoca el intérprete correspondiente.
5. Ajusta credenciales si hay bits `setuid/setgid`.
6. Guarda argumentos y entorno en espacio del kernel.
7. Libera el espacio de direcciones anterior.
8. Asigna uno nuevo y carga el programa.
9. Copia argumentos y entorno a la nueva pila.
10. Restaura manejadores de señales por defecto.
11. Inicializa el contexto hardware y apunta al inicio del programa.

Finalización de un proceso: `exit()`

Al terminar, un proceso llama a `exit()` y realiza las siguientes tareas:

- Desactiva señales.
- Cierra archivos abiertos.
- Libera entradas en tablas de nodos virtuales.
- Guarda estadísticas de uso y estado de salida de su `proc`.
- Cambia su estado a zombie (SZOMB).
- Sus hijos pasan a ser hijos de `init`.
- Libera espacio de direcciones, `u_area` y espacio de intercambio.
- Envía SIGCHLD al proceso padre
- Si el padre está esperando, se despierta.
- Realiza un cambio de contexto (switch).

El proceso sigue existiendo como zombie hasta que su padre use `wait()`.

Esperar la terminación de un hijo: `wait()`

Cuando un proceso desea saber cómo ha terminado su hijo, puede invocar `wait()` o una de sus variantes:

- `wait()` se bloquea hasta que termina cualquier hijo.
- Retorna el PID del hijo terminando y su código de salida.

- Libera la estructura `proc` del hijo.

Variantes como `waitpid()`, `waitid()`, `wait3()` o `wait4()` permiten opciones como:

- `WNOHANG`: no espera si el hijo aún no ha terminado.
- `WUNTRACED`: informa también si el hijo ha sido detenido.
- `WCONTINUED`: informa si un hijo detenido ha continuado.
- `WNOWAIT`: no libera la estructura `proc`

Sin una llamada a `wait()`, el proceso queda como zombie. para evitarlo se puede usar el flag `SA_NOCLDWAIT` en `sigaction`.

Interpretación del valor devuelto

El valor pasado como parámetro a `wait (int *stat_loc)` contiene:

- Si el proceso terminó con `exit()`: los 8 bits menos significativos son 0, y los siguientes 8 contienen el valor devuelto.
- Si terminó por una señal: 8 bits menos significativos son 0, y los inferiores contienen el número de la señal.
- Si fue detenido: contiene una marca especial (`WSTOPFLG`) y el número de señal que lo detuvo.

Planificación de procesos

En un sistema operativo multiprogramado, varios procesos, y en muchos casos, hilos, compiten constantemente por el uso de la CPU. Cada proceso está compuesto por una sucesión de ráfagas de CPU y ráfagas de entrada/salida (E/S), comenzando y finalizando siempre con una ráfaga de CPU.

El componente encargado de decidir cuál de los procesos preparados para ejecutarse obtiene el control de la CPU es el **planificador**

Algoritmos de planificación

Existen dos tipos principales de algoritmos de planificación, depende cómo gestionan la asignación de la CPU:

- **Algoritmos no expropiativos**: el proceso que está ejecutandose en la CPU permanece en ella hasta que finaliza su ráfaga de CPU de forma voluntaria. Esto puede deberse, por ejemplo, al inicio de una operación de E/S, a la espera del fin de un proceso hijo o simplemente porque el proceso ha terminado.
- ****Algoritmos expropiativos**: el planificador puede interrumpir al proceso en ejecución y retirarlo de la CPU, incluso aunque no haya terminado su ráfaga. Esto permite una mayor

capacidad de respuesta, especialmente en entornos interactivos.

Tipos de planificadores

En un sistema operativo pueden intervenir distintos tipos de planificadores, según el momento y el nivel del sistema en que actúan:

- **Planificador a corto plazo:** selecciona, entre los procesos listos para ejecutarse, cuál accede a la CPU. Es el que se invoca con más frecuencia.
- **Planificador a medio plazo:** presente en sistemas con intercambio, swapping, decide qué procesos que han sido sacados de memoria serán reintroducidos en ella.
- **Planificador a largo plazo:** en sistemas por lotes, se encarga de decidir qué procesos en la cola de espera del dispositivo de spool se cargarán en memoria principal. Controla el grado de multiprogramación del sistema.

Objetivos de la planificación

Los objetos de un planificador pueden variar dependiendo del entorno en el que se aplique. Algunos de los más relevantes son:

Sistemas por lotes (batch)

- Suele utilizarse planificación no expropiativa.
- El planificador a largo plazo es fundamental.
- Se prioriza la **eficiencia** y un alto **rendimiento**, es decir, ejecutar el mayor número de trabajos posible por unidad de tiempo.
- Otros objetivos incluyen minimizar el **tiempo de retorno** y maximizar el **uso de la CPU**

Sistemas interactivos

- Predominan los algoritmos expropiativos, especialmente con planificación por **compartición de tiempo**.
- El objetivo principal es que todos los procesos reciban CPU en un intervalo razonable, favoreciendo una **buena capacidad de respuesta**.
- También se busca la **proporcionalidad**, es decir, que tareas sencillas obtengan tiempos de respuesta cortos.
- Se intenta maximizar el número de usuarios activos interactuando con el sistema.

Sistemas en tiempo real

- Algunos procesos tienen **restricciones temporales estrictas** que deben cumplirse.
- Se suele aplicar planificación basada en **prioridades**, asignando las más altas a los procesos con requerimientos más exigentes.

- Los objetivos fundamentales son la **fiabilidad** y la **previsibilidad**.

Criterios típicos de evaluación

Aunque cada sistema tiene sus propios objetivos específicos, existen ciertos criterios comunes:

- **Equidad:** todos los procesos deben tener acceso razonable a la CPU.
- **Política:** se deben respetar criterios previamente definidos.
- **Balance:** lograr que las distintas partes del sistema trabajen con cargas similares.

En sistemas por lotes

- **Rendimiento:** número de trabajos completados por unidad de tiempo. Cuanto mayor, mejor.
- **Tiempo de retorno:** tiempo total desde que se envía un trabajo hasta que termina. Se busca minimizarlo.
- **Tasa de uso de CPU:** mantener la CPU ocupada el mayor tiempo posible.

En sistemas de tiempo compartido

- **Tiempo de respuesta:** tiempo desde que el usuario lanza una tarea hasta que recibe una primera respuesta.
- **Proporcionalidad:** las tareas simples deben responder rápidamente.
- **Maximización de usuarios activos:** permitir que muchos usuarios trabajen de forma interactiva.

En sistemas en tiempo real

- **Fiabilidad:** evitar pérdidas de datos y cumplir límites temporales.
- **Previsibilidad:** asegurar que el comportamiento temporal del sistema sea constante y controlable.

Evaluación de un algoritmo de planificación

La evaluación de un algoritmo de planificación busca determinar cuál se adapta mejor a las necesidades de un sistema. Dado que los objetivos pueden variar, existen tres enfoques principales:

Modelos deterministas

Se parte de una carga de trabajo concreta y se analiza cómo se comporta el sistema. Por ejemplo, se puede calcular el tiempo medio de retorno, el rendimiento, etc.

Ventajas:

- Simples de aplicar
- Ofrecen resultados exactos.

Desventajas:

- Pueden ser engañosos si la carga de trabajo no representa adecuadamente la realidad del sistema

Modelos no deterministas

En muchos sistemas no se puede predecir ni el momento de llegada ni la duración de los procesos. Por tanto, se utilizan funciones de distribución de probabilidad para modelar estos factoes.

Con estas distribuciones se pueden estimar valores medidos como:

- Tiempo de espera.
- Tiempo de retorno.
- Uso de CPU.

El sistema se modela como un conjunto de **servidores con colas de espera**, donde la CPU es uno de estos servidores y cada dispositivo de E/S puede ser otro.

Si se conoce la tasa de llegada de procesos y su duración, se pueden calcular métricas como:

- Carga del servidor.
- Longitud media de las colas.

Simulación

Otra opción consiste en simular el comportamiento del sistema con un conjunto de procesos.

Ventajas:

- Proporcionan una visión bastante realista del comportamiento del algoritmo.

Desventajas:

- Elevado coste computacional.

Implementación

Consiste en implementar directamente el algoritmo en un sistema real para observar su rendimiento.

Ventajas

- Los datos obtenidos son reales

Desventajas

La simple presencia de un nuevo algoritmo puede alterar el comportamiento de los usuarios y, por tanto, distorsionar los resultados.

Medición temporal del procesos

Para evaluar el comportamiento de los algoritmos, se utilizan varias métricas relacionadas con el tiempo:

- **Tiempo de retorno:** es el tiempo desde que el proceso comienza hasta que termina.
- **Tiempo de espera:** se obtiene restando al tiempo de retorno el tiempo de CPU y el de E/S.
- **Índice de servicio:** proporción entre el tiempo de servicio y el tiempo total de retorno:

Planificación no expropiativa

FCFS: First-Come, First-Served

Este algoritmo es uno de los más simples en la planificación de procesos. Su funcionamiento se basa en una estructura de tipo FIFO, es decir, los procesos se ejecutan en el orden en que llegan.

Ventajas:

- Es fácil de implementar, ya que basta con una cola sencilla.
- Resulta justo en el sentido de que los procesos se ejecutan por orden de llegada.

Inconveniente principal:

Tiene un bajo rendimiento en términos de **throughput** debido al fenómeno llamado "efecto convoy", que ocurre cuando un proceso requiere mucho tiempo de CPU bloquea a muchos procesos más cortos que llegan después. Por ejemplo, un proceso intensivo en CPU puede retrasar a varios procesos que sólo necesitan hacer operaciones de entrada/salida.

SJF: Shortest Job First

El algoritmo de planificación SJF selecciona el proceso con la ráfaga de CPU más corta entre los que están listos para ejecutarse. Este enfoque reduce al mínimo el tiempo medio de retorno **cuando todos los procesos llegan al mismo tiempo-**

Ventajas:

- Produce tiempos de retorno muy bajos, siendo óptimo en el caso ideal en que se conocen todas las ráfagas de CPU de antemano.

Limitaciones

- Es un algoritmo teórico, ya que en la práctica no se conoce con certeza cuánto durará la ráfaga de CPU de un proceso.
- Al ser no expropiativo, los resultados dependen mucho del orden de llegada de los procesos.

SPN: Shortest Process Next

Suele implementarse mediante una **estimación de la próxima ráfaga de CPU**, usando la fórmula:

$$t_{+1} = t + (1 - \alpha) \cdot t_{est}$$

Donde:

- t_{+1} es la próxima estimación.
- t es la duración real de la última ráfaga.
- t_{est} es la estimación anterior.
- α es un factor de ajuste entre 0 y 1.

Planificación por prioridades no expropiativa

SJF puede considerarse un caso particular de planificación por prioridades, en el que las prioridades se asignan según la duración estimada de la ráfaga de CPU.

Tipos de prioridades:

- **Internas:** calculadas por el sistema operativo, basadas en características como uso de CPU, memoria, archivos abiertos, etc.
- **Externas:** asignadas por el usuario o el administrador del sistema.
- **Mixtas:** combinación de ambas.

Además, las prioridades pueden ser:

- **Estáticas:** no cambian a lo largo del tiempo.
- **Dinámicas:** el sistema las modifica automáticamente según el comportamiento de los procesos.

Problema importante: la **iniciación**, donde los procesos con baja prioridad pueden no llegar a ejecutarse. Esto suele mitigarse mediante:

- **Aging:** aumento progresivo de la prioridad de un proceso cuanto más tiempo pase esperando.
- Prioridades basadas en el cociente entre tiempo de espera y uso de CPU.

Planificación Expropiativa

Prioridades Expropiativas

A diferencia de la versión no expropiativa, aquí **un proceso puede ser interrumpido** si otro con prioridad superior se vuelve elegible.

- El proceso que estaba ejecutándose entra en estado de *ready*, y el nuevo proceso toma la CPU.
- Las mismas clasificaciones de prioridad (interna, externa, dinámica...) se aplican aquí.
- El rendimiento de este método depende de cómo se asignen las prioridades.

SRTF: Shortest Remaining Time First

Este es el equivalente expropiativo de SJF. Cuando un nuevo proceso llega, su tiempo estimado de ráfaga se compara con el tiempo restante del proceso que está en ejecución. Si el nuevo proceso tiene una ráfaga más corta, interrumpe al actual. Este algoritmo también requiere conocer, o estimar, las ráfagas de CPU.

Round-Robin

Round-Robin asigna a cada proceso un **quantum** de tiempo máximo en CPU. Si un proceso agota su quantum, se interrumpe su ejecución, se coloca al final de la cola de procesos listos, y el siguiente proceso en la cola toma el control.

Ventajas

- Fácil de implementar.
- Muy justo: todos los procesos obtienen su "porción" de tiempo.

Desventajas

- Si el quantum es muy pequeño, se generan muchos cambios de contexto, lo que desprecia recursos.
- Si es demasiado grande, el algoritmo se comporta como FCFS.

Regla práctica: se obtiene buenos resultados si el **80% de las ráfagas de CPU son más cortas que el quantum**. Valores típicos oscilan entre **20 y 50 ms**.

Multilevel Queue

Este algoritmo extiende la planificación por prioridades dividiendo los procesos en diferentes colas según su prioridad. Cada cola puede tener su propio algoritmo de planificación.

- Para evitar inanición, se emplean **prioridades dinámicas**, permitiendo que un proceso cambie de cola.

Planificación en Sistemas de Tiempo Real

En sistemas de tiempo real, el tiempo de respuesta es fundamental. Los estímulos generados por dispositivos físicos requieren respuestas dentro de un tiempo limitado.

Tipos:

- **Hard real-time:** todos los límites de tiempo deben cumplirse.
- **Soft real-time:** pueden tolerarse algunos retrasos ocasionales.

Los sistemas de tiempo real suelen gestionar **eventos periódicos** y **eventos no periódicos**.

Condición de planificabilidad:

Un sistema con m flujos periódicos es planificable si: $\sum_{i=1}^m \frac{C_i}{T_i} \leq 1$

- T_i : periodo del flujo i .
- C_i : periodo del flujo i .

Planificación de Hilos

Un **hilo** es la unidad básica de ejecución de un proceso. Mientras que un proceso puede tener uno o más hilos, todos los hilos de un mismo proceso **comparten el espacio de direcciones** y los recursos del proceso.

Cada hilo posee su:

- Identificador.
- Contador de programa.
- Registros.
- Pila.

Ventajas

- Mayor capacidad de respuesta: si un hilo se bloquea, otros pueden seguir ejecutándose.
- Menor coste de creación y cambio de contexto que los procesos.
- Aprovecha arquitecturas multiprocesador.

La planificación se produce a dos niveles:

1. Se selecciona un proceso.
2. Dentro de ese proceso, se selecciona un hilo.

Planificación en Multiprocesadores

En sistemas con varios procesadores, la planificación se complica.

Tipos de planificación:

- **Asimétrica:** un procesador se encarga de toda la planificación, los demás sólo ejecutan.
- **Simétrica:** cada procesador realiza su propia planificación.

En algunos sistemas, los procesadores comparten la cola de procesos listos. Esto implica tener cuidado para que un mismo proceso no sea asignado a más de un procesador.

Afinidad de caché

Cuando un hilo se ejecuta durante mucho tiempo en un procesador, su caché contiene datos relevantes. Volver a ejecutar ese hilo en el mismo procesador mejora el rendimiento.

Algoritmos de afinidad:

1. Asignan grupos de hilos a cada procesador.
2. Cada procesador planifica dentro de su grupo.

Ventajas

- Aprovechamiento de la caché.
- Menor coste de migración.

Inconvenientes

- Riesgo de dejar procesadores sin carga.

Balanceo de carga

Para evitar desequilibrios:

- **Migración forzada:** el sistema fuerza la redistribución de procesos cuando detecta que algunos procesadores están sobrecargados.
- **Migración voluntaria:** los hilos o procesadores pueden moverse de forma controlada si hay beneficio.

Planificación de procesos en UNIX

Planificación tradicional en UNIX

El sistema tradicional de planificación en Unix emplea un modelo de prioridades expropiativo, en el que siempre se ejecuta el proceso con mayor prioridad. En este contexto, una prioridad más alta se representa con un número menor. Esta prioridad se ajusta dinámicamente dependiendo del comportamiento del proceso:

- Si un proceso utiliza mucho tiempo de CPU, su prioridad disminuye.
- Si un proceso permanece mucho tiempo en la cola de listos, su prioridad aumenta.

Cuando aparece un proceso con mayor prioridad que el que actualmente ocupa la CPU, el sistema lo interrumpe, salvo que el proceso actual esté ejecutándose en modo kernel. En ese caso, la interrupción se pospone hasta que vuelva al modo usuario, termine o se bloquee. Los procesos con la misma prioridad comparten la CPU en régimen de "round-robin".

Factores de Recalculación de Prioridad

Las prioridades en modo usuario se recalculan teniendo en cuenta dos factores:

- **El factor "nice"**: modificable mediante la syscall `nice()`, permite al usuario influir en la prioridad.
- **Uso reciente de CPU**: cuanto más CPU haya usado un proceso recientemente, menor será su prioridad.

La estructura `proc` del núcleo contiene varios campos relevantes para esta planificación:

- `p_cpu`: refleja el uso de CPU reciente.
- `p_nice`: almacena el valor de "nice".
- `p_usrpri`: prioridad en modo usuario, calculada a partir de `p_cpu` y `p_nice`
- `p_pri`: prioridad efectiva usada en la planificación.

En modo usuario, `p_pri` es igual a `p_usrpri`. Sin embargo, cuando un proceso despierta tras haber estado bloqueado, su prioridad (`p_pri`) se ajusta el motivo del bloqueo, lo que se conoce como **prioridad en modo kernel** o **prioridad de sueño**. Estas prioridades de kernel son más altas que las prioridades de usuario, con el fin de completar llamadas al sistema lo más rápido posible.

Recalculación de Prioridades en Modo Usuario

Cada "tick" de reloj, el sistema incrementa `p_cpu` para el proceso en ejecución.

Cada segundo, una rutina (`schedcpu()`) recalcula `p_cpu` y la prioridad de usuario:

- **En BSD:**
 - $$= \frac{2}{2+1}$$

- $s = + \left(\frac{1}{4} \right) + (2)$
- **En System V R3:**
- $= \frac{1}{2}$
- $s = + \left(\frac{1}{2} \right) +$

En ambos casos, PUSER es un valor base que asegura que las prioridades de usuario sean más bajas que las de kernel.

Implementación de la Planificación Tradicional

La planificación se implementa mediante un **array de colas multinivel**, típicamente 32 colas, cada una correspondiente a un rango de prioridades adyacentes. Cuando se recalcula la prioridad de un proceso, este se mueve a la cola apropiada.

- La función `switch()` carga el primer proceso de la primera cola no vacía.
- Cada 100 ms, `roundrobi()` fuerza el cambio al siguiente proceso en la misma cola.

Cambios de Contexto

Se produce un **cambio de contexto** en tres situaciones:

1. **Voluntario:** el proceso termina o se bloquea (`sleep()` o `exit()` llaman a `switch()`).
2. **Involuntario:**
 - Tras recalcular prioridades aparece un proceso con mayor prioridad.
 - Un proceso o rutina de interrupción despierta a un procesador de mayor prioridad.

En estos casos, si el sistema está en modo kernel, se utiliza un flag `runrun` para indicar que debe realizarse un cambio al volver al modo usuario.

Limitaciones de la planificación tradicional

Este modelo presenta varias **deficiencias**:

- No se adapta bien a sistemas con muchos procesos o procesadores.
- No garantiza una cuota mínima de CPU para un proceso o grupo.
- No asegura un tiempo de respuesta mínimo.
- Existe la posibilidad de **inversión de prioridad**.

Inversión de prioridad

Este fenómeno ocurre cuando un proceso de baja prioridad impide la ejecución de uno de mayor prioridad.

Planificación en System V Release 4

SVR4 introduce un modelo más sofisticado y extensible:

- Se separa la política de planificación del mecanismo de implementación.
- Permite implementar nuevas políticas.
- Incluye procesos en tiempo real.
- Evita la inversión de prioridad mediante herencia de prioridad.

Clases de planificación

Los procesos se agrupan en **clases de planificación**, cada una con su propia política. Algunas clases comunes:

- TS, Time Sharing, compartición de tiempo
- SYS, System, procesos del sistema
- RT, Real Time, procesos de tiempo real
- IA, Interactive, versión mejorada de TS

Consulta de clases en el sistema: `dispadmin -l`

Rango de prioridades

- 0-59: **TS**
- 60-99: **SYS**
- 100-159: **Real Time**

En la estructura `proc`, cada proceso tiene:

- `p_cid`: identificador de clase
- `p_clfuncs`: punteros a funciones específicas de la clase
- `p_clproc`: datos dependientes de la clase

Clase Time Sharing (TS) en SVR4

Los procesos en esta clase tienen **prioridades dinámicas** que se recalculan al djar la CPU:

- Si el proceso **agota** su tiempo asignado, su prioridad disminuye.
- Si el proceso se **bloquea** antes de agotarlo, su prioridad aumenta.

El quantum asignado depende de la prioridad: a mayor prioridad, menor tiempo asignado.

Campos relevantes:

- `ts_timeleft`: tiempo restante del quantum.
- `ts_cpupri`: parte del cálculo de prioridad gestionada por el sistema.

- `ts_upri` : parte controlada por el usuario mediante `priocntl()`.
- `ts_umdpr` : prioridad en modo usuario total (`ts_cpupri` + `ts_upri`).
- `ts_dispwait` : tiempo desde el inicio del quantum.

El sistema mantiene una tabla donde relaciona prioridad, quantum y reglas de actualización (`tqexp`, `slpret`, etc.).

Clase Real Time

- Prioridades fijas entre **100 y 159**.
- Quantums fijos para cada nivel de prioridad.
- No se recalculan automáticamente, sólo modificables mediante `priocntl()`.

Si un proceso real time está listo pero el proceso actual está en modo kernel, la preempción se retrasa hasta que el sistema:

- Vuelva al modo usuario.
- Se bloquee.
- Llega a un **punto de preempción** (en código kernel seguro para el cambio).

Los sistemas modernos como **Solaris** eliminan estos puntos de preempción mediante el uso extensivo de semáforos, permitiendo preempción total incluso en modo kernel.

Clase system

- Reservada para procesos especiales del sistema (`pageout`, `sched`, `fsflush`...)
- No disponible para todos los usuarios o instalaciones.
- Usa prioridades fijas entre 60 y 99.

Planificación en Linux

Linux distingue entre dos tipos de procesos:

1. Procesos de tiempo real:

- Prioridades fijas entre 1 y 99
- Algoritmos: **FIFO** y **Round-Robin**.
- Siempre se ejecuta el de mayor prioridad listo.

2. Procesos normales:

- Prioridad estática 0.
- Usan un algoritmo de prioridad dinámica con preempción.
- Sólo se ejecutan si no hay procesos de tiempo real listos.

Procesos en Unix: Ejecución en modo kernel

¿Qué significa ejecutar en modo kernel?

En los sistemas Unix, un proceso puede ejecutarse en dos modos: **modo usuario** y **modo kernel**. El **modo kernel** es un estado privilegiado en el que el sistema operativo tiene acceso completo al hardware en circunstancias específicas, ya que implica que el control pasa al núcleo del sistema operativo.

¿Cuándo se entra en modo núcleo?

Hay **tres situaciones** principales que provocan que un proceso cambie de modo usuario a modo núcleo:

1. Interrupción de dispositivo

Es un evento **asíncrono** que ocurre independientemente del proceso que esté corriendo. Un dispositivo externo necesita atención del sistema operativo. Esta interrupción puede suceder en cualquier momento: mientras se ejecuta código en modo usuario, en modo kernel o incluso durante otra rutina de interrupción

2. Excepción

Se trata de un evento **síncrono**, causado por el propio proceso en ejecución. Ejemplos comunes incluyen divisiones por cero, direcciones inválidas o instrucciones ilegales.

3. Llamadas al sistema

También es **síncrona** y ocurre cuando un proceso necesita explícitamente un servicio del sistema operativo, como leer un archivo, crear un proceso, etc.

Comportamiento del núcleo ante uno de estos eventos

Cuando ocurre cualquiera de estos eventos, el núcleo del sistema operativo toma el control del procesador:

- Guarda el contexto del procesador en su **pila del kernel**.
- Ejecuta la función correspondiente al evento.
- Una vez que termina el servicio, restaura el contexto del proceso y lo devuelve a su estado anterior, incluyendo su modo de ejecución.

Interrupciones

Las **interrupciones** son especialmente delicadas porque pueden suceder en cualquier momento. Para manejarlas adecuadamente:

- A cada interrupción se le asigna un **nivel de prioridad** llamado *IPL* o Interrupt Priority Level.

- Cuando llega una interrupción, su IPL se compara con el del proceso actual.
 - Si el IPL de la interrupción es **más alto**, se ejecuta inmediatamente el **manejador de interrupción**.
 - Si es **menor o igual**, se **pospone** hasta que el IPL del sistema sea lo suficientemente bajo.
- En general, todo el código de usuario y casi todo el código del núcleo se ejecuta con el IPL mínimo.
- En sistemas Unix tradicionales, el rango del IPL va de **0 a 7**. En sistemas BSD puede llegar hasta **31**.

Llamadas al sistema

Cuando se invoca una llamada al sistema como `read()`, `open()`, `fork()`, etc., en realidad no se accede directamente al núcleo. Lo que ocurre es lo siguiente:

1. **La biblioteca del sistema** proporciona una función wrapped, por ejemplo `read()`.
2. Esta función:
 - Coloca los parámetros de la llamada en la pila de usuario.
 - Inserta un número de servicio que identifica qué operación se solicita.
 - Ejecuta una **instrucción especial** (como `trap`, `chmk`, `int`, según la arquitectura), que hace dos cosas:
 - Cambia el modo de ejecución a **modo kernel**.
 - Transfiere el control a la rutina del núcleo que gestiona llamadas al sistema, una `syscall()`.
3. En `syscall()`
 - Se copian los argumentos a una estructura del núcleo (`u_area`).
 - Se guarda el contexto del proceso en la pila del núcleo.
 - Usando el número de servicio como índice, se accede a una tabla (`sysent[]`) que asocia cada número con una función del núcleo, por ejemplo `sys_read()`.
4. La función del kernel (`sys_read()`, etc.):
 - Presta el servicio solicitado.
 - Puede llamar a otras funciones del núcleo si es necesario.
 - Coloca los valores de retorno, o errores, en los registros apropiados.
 - Restaura el contexto del proceso y regresa al modo usuario.
 - Finalmente, el control vuelve a la función de la biblioteca y luego al programa del usuario.

Acceso a recursos

El modo núcleo proporciona mecanismos para proteger los recursos compartidos del sistema:

- Cuando un proceso entra en modo kernel, **no puede ser interrumpido por otro proceso**, lo que garantiza que puede manipular datos del kernel sin crear inconsistencias.
- Si un proceso necesita un recurso y este **ya está ocupado**, el proceso debe:
 1. Marcar el recurso como **deseado**
 2. Llamar a `sleep()`, que lo pone en estado de suspensión.
 3. Esta función invoca a `switch()`, iniciando un **cambio de contexto** y otro proceso toma el control.
- Cuando el recurso se libera:
 - Se marca como no ocupado.
 - Si había procesos esperando, se marcan como listos para ejecutar mediante `wakeup()`.
 - Todos estos procesos se colocan en la **cola de procesos ejecutables**.
- Sin embargo, no se garantiza que el primer proceso despertado sea el primero en obtener el recurso. Por ello, **debe comprobar de nuevo si está disponible** antes de usarlo.

Consideraciones avanzadas sobre interrupciones y concurrencia

Aunque un proceso en modo kernel no puede ser interrumpido por otro proceso, sí puede recibir **interrupciones de hardware**. Para ello:

- Si se está accediendo a datos que podrían ser utilizados por una **rutina de interrupción**, esta interrupción debe ser **desactivada temporalmente**, elevando el IPL.
- Esta elevación debe hacerse **con cuidado**:
 - Las interrupciones deben ser atendidas rápidamente.
 - El tiempo durante el cual se desactivan debe ser **mínimo**.
 - Elevar el IPL bloquea también **todas las interrupciones con prioridad inferior**.
- Si se desea que el núcleo sea **completamente preemptible**, es decir, que incluso el código del kernel pueda ser interrumpido por otros procesos, entonces se deben usar mecanismos más robustos como **semáforos** para proteger los datos del kernel.
- En sistemas multiprocesador, la complejidad aumenta, y se requieren **mecanismos de sincronización más sofisticados** para asegurar la coherencia de los datos compartidos.

Procesos en UNIX: señales

¿Qué son las señales?

En los sistemas Unix las señales son una forma de comunicación entre el kernel y los procesos. Se utilizan para **notificar la ocurrencia de eventos asíncronos**, es decir, eventos que no se producen como consecuencia directa del flujo de ejecución del programa.

Por ejemplo, si un usuario pulsa la combinación de teclas **Ctrl+c**, el kernel envía la señal **SIGINT** al proceso en primer plano para interrumpir su ejecución. Otro ejemplo: si se cae una

línea de comunicación, el sistema puede enviar la señal **SIGHUP**.

No sólo el kernel puede enviar señales; también los procesos pueden enviarse señales entre ellos mediante la llamada al sistema `kill`.

Comportamiento de los procesos ante señales

Los procesos pueden reaccionar de diferentes maneras ante la recepción de una señal:

- **Terminar el proceso.**
- **Ignorar la señal**, haciendo que no ocurra nada.
- **Definir un manejador de señal**, una función que se ejecuta cuando la señal es recibida.

Cuando el kernel detecta que debe enviar una señal, lo que realmente hace es **marcar un bit** en una estructura del proceso `proc`. La reacción se ejecuta sólo cuando el proceso **retorna al modo usuario**:

- Si el proceso está en modo kernel, la señal se aplaza hasta volver al modo usuario.
- Si el proceso está **bloqueado** esperando algo:
 - Si la espera es **interrumpible**, se interrumpe la llamada al sistema, retorna -1 y deja `errno` en EINTR.
 - Si la espera no es interrumpible, la señal se trata cuando el proceso regresa a modo usuario tras completar la espera.

Señales en System V Release 2

Esta versión del sistema operativo incluía **15 señales disponibles y algunas llamadas al sistema relacionadas:

- `kill(pid_t pid, int sig)`: permite a un proceso enviar una señal a otro.
- `signal (int sig, void (*handler)(int))`: asocia una señal a un manejador. El manejador puede ser:
 - `SIG_DFL`: la acción por defecto de la señal.
 - `SIG_IGN`: la señal se ignora.
 - Una **función definida por el usuario**, que recibe como parámetro el número de señal.

El sistema mantiene un **array en el área de proceso (`u_area`)** indexado por número de señal, que contiene los manejadores correspondientes o indica si se debe ignorar o aplicar la acción por defecto.

Cuando el proceso va a volver al modo usuario y tiene señales pendientes:

1. El kernel limpia el bit correspondiente.

2. Si la señal se ignora, no pasa nada.
3. Si hay un manejador definido:
 - Se crea un nuevo contexto en la pila del proceso.
 - Se establece el PC en la dirección del manejador.
 - La señal se restablece a su acción por defecto.

Los manejadores **no son permanentes**. Si se recibe la misma señal de nuevo, ya no se llama al manejador, salvo que se reinstale.

```
#include <signal.h>
void manejador()
{
    printf("Se ha pulado control-C\n");
}
int main()
{
    signal(SIGINT, manejador);
    while(true); //bucle infinito
}
```

En este caso, al pulsar Ctrl+C una vez se ejecuta el manejador, pero si se pulsa otra vez antes de que `signal(SIGINT, manejador);` se reinstale, el programa termina.

Versión permanente

```
#include <signal.h>
void manejador()
{
    printf("Se ha pulado control-C\n");
    signal(SIGINT, manejador); //Reinstala el manejador
}
int main()
{
    signal(SIGINT, manejador);
    while(true);
}
```

System V Release 3: Señales fiables

System V R3 introduce las **señales fiables**, donde:

- Los manejadores se vuelven permanentes.
- El manejador de una señal **se ejecuta con dicha señal bloqueada**.

- Se puede **bloquear y desbloquear señales** manualmente.
- El estado de las señales se almacena en la estructura `proc`.

Nuevas llamadas al sistema

- `sigset(int señal, void (*handler)(int))`: instala un manejador permanente que no puede ser interrumpido por la misma señal.
- `sighold(int señal)`: bloquea una señal
- `sigrelse(int señal)`: desbloquea una señal
- `sigpause(int señal)`: desbloquea la señal indicada y bloquea el proceso hasta que reciba una señal.

Señales en BSD (Berkeley Software Distribution)

BSD mejora el control de señales con varias características:

- **Bloqueo de señales en grupo**, mediante máscaras de bits.
- Si una señal **interrumpe una llamada al sistema**, esta se reinicia automáticamente.

Nuevas llamadas

- `sigsetmask(int mask)`: define el conjunto de señales bloqueadas.
- `sigblock(int mask)`: añade señales al conjunto bloqueado.
- `sigpause(int mask)`: establece la máscara y suspende el proceso.
- `sigvec(int sig, struct sigvec *vec, struct sigvec *ovec)`: instala el manejador para una señal.
- `sigstack(struct sigstack *ss, struct sigstack *oss)`: define una **pila alternativa** para los manejadores.

System V Release 4: Estándar moderno

System V R4 integra y mejora todas las funcionalidades anteriores. Es la base de la mayoría de sistemas Unix actuales.

Llamadas principales

- `sigprocmask(int how, const sigset_t *set, sigset_t *oldset)`: gestiona la **máscara de señales bloqueadas** del proceso.
 - `how` puede ser:
 - `SIG_BLOCK`: añade señales al conjunto bloqueado.
 - `SIG_UNBLOCK`: elimina señales del conjunto bloqueado.
 - `SIG_SETMASK`: reemplaza el conjunto entero.
 - `oldset` permite recuperar el conjunto anterior.

- Manipulación de conjuntos de señales (`sigset_t`):

```
int sigemptyset(sigset_t *set); // Vacía el conjunto
int sigfillset(sigset_t *set); // Llena el conjunto con todas las señales
int sigaddset(sigset_t *set, int signum); // Añade una señal
int sigdelset(sigset_t *set, int signum); // Elimina una señal
int sigismember(const sigset_t *set, int signum); // Verifica si una señal
está en el conjunto
```

- `sigsuspend(const sigset_t *mask)`: establece una máscara y **suspende el proceso** hasta recibir una señal que no esté bloqueada ni ignorada.
- `sigpending(sigset_t *set)`: consulta si hay señales pendientes.
- `sigaltstack(const stack_t **ss, stack_t *oss)`: define una pila alternativa para los manejadores.
- `sigsendset(procset_t *psp, int sig)` y `sigsend(idtype_t idtype, id_t id, int sig)`: permiten enviar señales con más opciones que kill.

sigaction: Control completo del comportamiento de las señales

Una de las llamadas más potentes es `sigaction`, que permite definir de forma muy precisa cómo debe reaccionar un proceso a una señal.

```
int sigaction(int signum, const struct sigaction *act, struct sigaction
*oldact);
```

La estructura de `sigaction` contiene:

- `sa_handler`: puede ser `SIG_DFL`, `SIG_IGN` o la dirección de un manejador.
- `sa_mask`: conjunto de señales a bloquear durante la ejecución del manejador.
- `sa_flags`: modificadores del comportamiento, entre ellos:
 - `SA_ONSTACK`: el manejador se ejecuta en la pila alternativa.
 - `SA_RESETHAND`: el manejador es no permanente.
 - `SA_NODEFER`: la señal no se bloquea durante la ejecución del manejador.
 - `SA_RESTART`: si una señal interrumpe una llamada al sistema, esta se reinicia automáticamente.
 - Otros: `SA_SIGINFO`, `SA_NOCLDWAIT`, `SA_NOCLDSTOP`, `SA_WAITSIG`.

Comunicación entre procesos

En los sistemas Unix, existen diversos mecanismos para que los procesos puedan comunicarse entre sí. Estos mecanismos forman parte fundamental del sistema operativo y permiten que los procesos intercambien información o se coordinen en sus acciones. Los principales métodos de comunicación entre procesos (Inter Process Communication) son:

- Tuberías o pipes.
- Memoria compartida.
- Semáforos.
- Colas de mensajes.

Pipes

Los pipes son un método sencillo y eficiente para permitir la comunicación entre procesos. Funcionan como archivos temporales que pueden ser utilizados para enviar información de un proceso a otro.

Para crear una tubería se utiliza la llamada al sistema `pipe()`:

```
#include <unistd.h>
int pipe(int fildes[2]);
```

Esta función devuelve dos descriptores de archivo:

- `fildes[0]`: extremo de **lectura**
- `fildes[1]`: extremo de **escritura**

En algunos sistemas, ambos extremos pueden ser usados tanto para lectura como para escritura, pero en la mayoría se sigue usando el estándar histórico, se utilizan de forma esperada: uno exclusivamente para leer y otro para escribir.

Funcionamiento

- Cuando **no hay datos** en la tubería, una llamada a `read()` se bloquea hasta que haya algo disponible.
- Cuando la tubería está **llena**, una llamada `write()` se bloquea hasta que haya espacio.
- Los datos se **descartan automáticamente** una vez han sido leídos.

Memoria compartida

La memoria compartida permite que varios procesos accedan directamente a una región común de memoria. Este mecanismo es más rápido que otros métodos, ya que evita la copia de datos entre procesos.

Paso 1: Obtener un bloque de memoria

Para usar memoria compartida, primero se debe obtener, o crear, un segmento de memoria con `shmget()`:


```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int shmget(key_t key, size_t size, int shmflg);
```

- `key`: clave numérica que identifica la región.
- `size`: tamaño de la región de memoria.
- `shmflg`: permisos y banderas.

Uso de las banderas:

- 0: devuelve el identificador si existe; error si no.
- `IPC_CREAT`: crea el recurso si no existe.
- `IPC_CREAT | IPC_EXCL`: error si ya existe, útil para evitar reutilizar recursos por error.

Paso 2: Asociar la memoria al proceso

Una vez creado el segmento, se debe "adjuntar" al espacio de direcciones de memoria con

`shmat()`:

```
void *shmat(int shmid, const void *shmaddr, int flg);
```

- `shmid`: identificador del segmento, devuelto por `shmget`.
- `shmaddr`: dirección virtual, usar NULL para que el sistema lo asigne.
- `flg`: opciones como `IPC_RDONLY`

Devuelve un puntero a la región de memoria compartida que el proceso puede usar para acceder directamente a los datos.

Paso 3: Desasociar la memoria.

Cuando ya no se necesita, el segmento debe ser "desmontado" con `int shmdt(char *shmaddr);`

Paso 4: Control del recurso

Para operaciones avanzadas como eliminar la memoria del sistema o consultar información, se usa

```
shmctl(): int shmctl(int shmid, int cmd, struct shmid_ds *buf);
```

Comandos como `IPC_RMID` (eliminar), `IPC_STAT`, `IPC_SET`, permiten administrar el recurso.

```
void *ObtenerMemoria(key_t clave, off_t tam, int crear)
{
    int id;
    void *p;
```

```

int flags = 0666;
if(crear)
    flags |= IPC_CREAT | IPC_EXCL;
if((id = shmget(clave, tam, flags)) == -1)
    return NULL;
if((p = shmat(id, NULL, 0)) == (void *) -1)
{
    if(crear)
        shmctl(id, IPC_RMID, NULL);
    return NULL;
}
return p;
}

```

Semáforos

Los semáforos permiten sincronizar el acceso a recursos compartidos entre múltiples procesos. Unix proporciona una interfaz para manejar **arrays de semáforos**, especialmente útil cuando se usan junto a memoria compartida.

Paso 1: crear o acceder a un array de semáforos

```
int semget(key_t key, int nsems, int semflg);
```

- `key`: clave del recurso.
- `nsems`: número de semáforos en el array.
- `semflg`: permisos y banderas como en `shmget`.

Paso 2: Realizar operaciones sobre semáforos

```
int semop(int semid, struct sembuf *sops, size_t nsops);
```

Se pueden realizar varias operaciones con un array de estructuras `sembuf`

```

struct sembuf{
    ushort_t sem_num; // número de semáforo
    short sem_op;      // operación a realizar
    short sem_flg;     // flags (SEM_UNDO, IPC_NOWAIT)
};

```

`sem_op` puede ser:

- negativo: espera a que el valor sea suficiente y lo decrementa.
- cero: espera a que el valor sea 0.
- positivo: incrementa el valor del semáforo.

Paso 3: Control del recurso

```
int semctl(int semid, int semnum, int cmd, ...);
```

Permite eliminar, consultar o modificar semáforos.

El cuarto argumento puede ser un union semun, con distintas posibilidades:

```
union semun{
    int val;
    struct semid_ds *buf;
    ushort *array;
};
```

Colas de mensajes

Las colas de mensajes permiten el envío de bloques de datos estructurales entre procesos. A diferencia de los pipes, aquí los mensajes pueden tener un **tipo**, lo que permite una mayor flexibilidad.

Paso 1: Crear o acceder a una cola de mensajes

```
int msgget(key_t key, int msgflg);
```

Como en los recursos anteriores, se usa una clave y unas banderas.

Paso 2: Enviar y recibir mensajes

```
int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);
int msgrcv(int msqid, void *msgp, size_t msgsz, long msgtyp, int msgflg);
```

- `msgsz`: tamaño del mensaje sin incluir el campo de tipo.
- `msgtyp`: tipo de mensaje a recibir:
 - 0: el primero de la cola.
 - n: el primero con tipo n.
 - -n: el primero con tipo menor o igual a n.
- `msgflg`: permite operaciones como `IPC_NOWAIT`, `MSG_NOERROR`.

Paso 3: Control de la cola

```
int msgctl(int msqid, int cmd, struct msqid_ds *buf);
```

Con `msgctl()` se puede eliminar una cola (`IPC_RMID`) o consultar/modificar su estado (`IPC_STAT`, `IPC_SET`).

Concurrencia

¿Qué es un spooler?

El término *spool* proviene de "Simultaneous Peripheral Operations On-Line", y se refiere a una técnica que permite gestionar múltiples trabajos entrada/salida de formas simultánea, almacenándolos temporalmente en una cola antes de ser procesados. En el caso de las impresoras, un *spooler* permite que varios procesos puedan enviar documentos a imprimir sin que se interfieran directamente entre ellos. El demonio de impresión (*printer daemon*) consulta el *spooler* y se encarga de imprimir los archivos en el orden adecuado. Imaginemos una estructura donde el *spooler* se comporta como un buffer. En este contexto, tenemos dos variables compartidas importantes:

- **in**: indica la proxima posición libre del *spooler*.
- **out**: indica la siguiente posición que el demonio de impresión debe imprimir.

Condiciones de carrera

Este fenómeno se conoce como **condición de carrera**, y aparece cuando varios procesos acceden a recursos compartidos sin una coordinación adecuada. El resultado del programa depende entonces del orden en que se intercalan las instrucciones de los procesos, lo cual introduce comportamientos inesperados y difíciles de depurar, ya que los errores no siempre se manifiestan.

Sección crítica y su control

¿Cómo evitamos las condiciones de carrera?

El primer paso es identificar las **secciones críticas**, es decir, aquellas partes del código donde se accede o modifica un recurso compartido. Si varios procesos ejecutan estas secciones de forma simultánea, pueden producirse errores como el descrito anteriormente.

Para evitarlo, cualquier solución al problema de la sección crítica debe garantizar:

- **Exclusión mutua**: sólo un proceso puede estar ejecutando su sección crítica en un momento dado.
- **Progreso**: si ningún proceso está en su sección crítica, no debe impedirse que otros procesos accedan a ella.
- **Espera limitada**: ningún proceso debe esperar indefinidamente para acceder a su sección crítica.
- **Independencia del número o velocidad de los procesadores**: la solución debe funcionar correctamente sin depender del entorno hardware.

Atomicidad

Una herramienta clave para implementar soluciones a estos problemas es la **atomicidad**. Una secuencia de instrucciones se considera **atómica** si el sistema operativo garantiza que no habrá cambios de contexto durante su ejecución. Es decir, se ejecuta de manera indivisible.

Gracias a esto, se han propuesto varias soluciones clásicas al problema de la sección crítica, como:

- **semáforos**: propuestos por Dijkstra en 1965
- **monitores**: introducidos por Hansen en 1973 y Hoare en 1974

La solución más común en sistemas UNIX es el uso de **semáforos**.

Semáforos

Un **semáforo** es una variable entera que sólo puede ser manipulada a través de dos operaciones atómicas:

1. P(sem) o Wait(sem): `wait` hasta que `sem > 0`; luego `sem--`;
Si el valor del semáforo es cero, el proceso queda **bloqueado**, sin consumir CPU.
2. V(sem) o Signal(sem): `sem++`;

Si hay procesos esperando en el semáforo, uno de ellos es **despertado** por el sistema operativo, que se encarga de reanudarlo de forma atómica.

Un **semáforo binario** es un caso particular donde el semáforo puede tener los valores 0 o 1, lo cual lo convierte en una herramienta ideal para garantizar exclusión mutua.

Solución al problema del spooler con semáforo

Podemos resolver el problema original del *spooler* utilizando un semáforo binario mutex, con valor inicial igual 1. Esto asegura que sólo un proceso pueda entrar en su sección crítica en un momento dado:

```
void printFile(char *fname)
{
    wait(mutex); // Entrada sección crítica
    load shared var "in" into reg;
    spooler[reg] = fname;
    write reg + 1 in shared var "in"
    signal(mutex); // Salida sección crítica
}
```

De igual manera, el demonio de impresión, que accede al *spooler* para imprimir archivos, también debe proteger su acceso usando el mismo semáforo mutex.

1. [Introducción a sistemas operativos](#)

2. [Ficheros](#)

3. [Memoria](#)

4. [Procesos](#)

5. [Entrada Salida](#)